BÀI TẬP TÌM KIẾM TRONG AI

Họ và tên	Trần Trọng Nghĩa
MSSV / Lớp	3123410236 / DCT1235

Mục tiêu học tập

• Mô phỏng các thuật toán BFS, DFS và Iterative Deepening.

• Biểu diễn bài toán Tháp Hà Nội dưới dạng bài toán tìm kiếm.

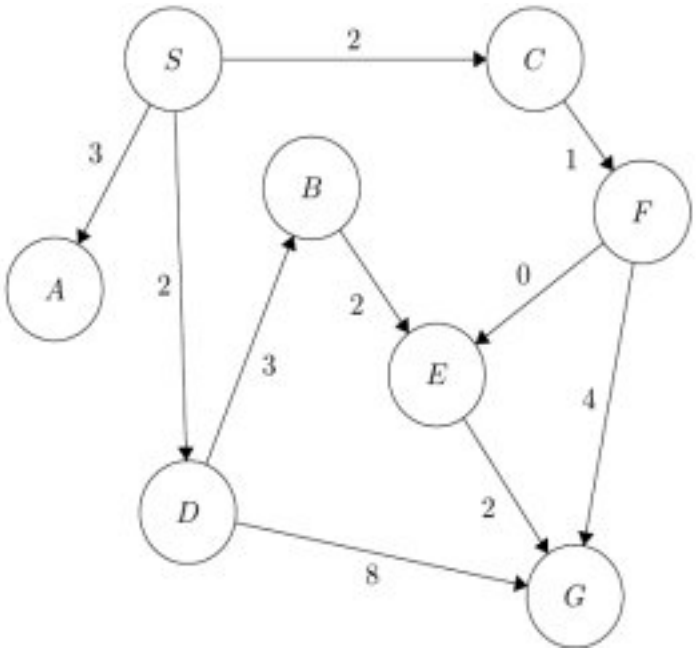
• Thiết kế và phân tích heuristic cho A*.

• So sánh DFS, BFS, UCS, Greedy Search và A* trên cùng một bài toán.

• Phân tích các phát biểu lý thuyết về heuristic, A*, Minimax và Expectimax.

Bài 1. Các thuật toán tìm kiếm

Cho đồ thị có trạng thái bắt đầu S và trạng thái đích G như hình dưới đây.



Hình 1. Đồ thị dùng cho các thuật toán tìm kiếm

Yêu cầu: Với mỗi thuật toán, hãy ghi thứ tự các nút được đưa vào explored set, đường đi được trả về và tổng chi phí của đường đi. Nếu có hòa, phá hòa theo thứ tự alphabet của trạng thái cuối cùng trong đường đi.

1.1. Breadth-First Search (BFS)

Nội dung	Kết quả
Explored Set	S, A, C, D, F, B, G
Path	S -> D -> G
Total Cost	10

1.2. Depth-First Search (DFS)

Nội dung	Kết quả
Explored Set	S, A, C, F, E, G
Path	S -> C -> F -> E -> G
Total Cost	5

1.3. Iterative Deepening Search (IDS)

Giới hạn độ sâu	Các nút được duyệt
0	S
1	S, A, C, D
2	S, A, C, F, E, D, B, G
...	Tìm thấy G ở giới hạn độ sâu 2

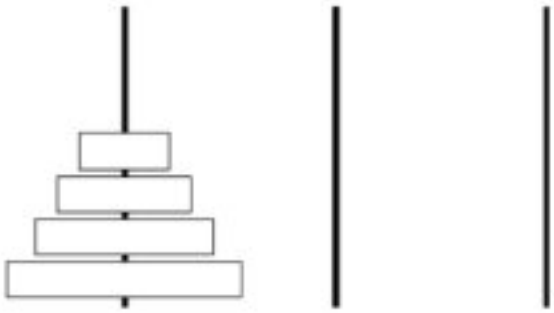
Path cuối cùng: S -> C -> F -> G

Total Cost: 7

Câu hỏi thảo luận: BFS sử dụng hàng đợi FIFO (Queue), duyệt theo chiều rộng từng tầng nên đảm bảo tìm được đường đi ngắn nhất (ít cạnh nhất). DFS sử dụng ngăn xếp LIFO (Stack), duyệt sâu hết một nhánh trước nên không đảm bảo tối ưu và có thể bị lặp vô tận nếu đồ thị có chu trình.

Bài 2. Tháp Hà Nội - Tower of Hanoi

Bài toán gồm n đĩa có kích thước khác nhau và ba cọc. Ban đầu các đĩa được xếp trên một cọc theo thứ tự từ lớn đến nhỏ. Mỗi lần chỉ được di chuyển một đĩa và không được đặt đĩa lớn lên trên đĩa nhỏ. Mục tiêu là chuyển toàn bộ đĩa sang cọc thứ ba.



Hình 2. Trạng thái ban đầu của bài toán Tháp Hà Nội

2.1. Biểu diễn trạng thái

Hãy đề xuất cách biểu diễn một trạng thái của bài toán. Biểu diễn phải cho biết mỗi đĩa đang ở cọc nào và đủ để xác định các hành động hợp lệ.

Trạng thái được biểu diễn bằng một tuple/mảng gồm n phần tử S = (p1, p2, ..., pn), trong đó pi ∈ {1, 2, 3} là vị trí cọc của đĩa i (đĩa 1 là nhỏ nhất, đĩa n là lớn nhất). Do các đĩa trên cùng một cọc luôn được sắp xếp theo thứ tự lớn ở dưới, nhỏ ở trên, cách biểu diễn này duy nhất và xác định hoàn toàn cấu hình các cọc.

2.2. Kích thước không gian trạng thái

Mỗi đĩa có 3 sự lựa chọn vị trí cọc độc lập. Với n đĩa, tổng số trạng thái hợp lệ trong không gian trạng thái là 3^n.

2.3. Trạng thái bắt đầu

Hãy viết trạng thái khởi đầu S0 theo cách biểu diễn đã chọn.

S0 = (1, 1, ..., 1) (tất cả n đĩa đều nằm ở cọc 1).

2.4. Các hành động hợp lệ

Hành động Move(d, c_from, c_to): Di chuyển đĩa d từ cọc c_from sang cọc c_to. Điều kiện hợp lệ: (1) Đĩa d phải là đĩa trên cùng (nhỏ nhất) ở cọc c_from. (2) Cọc c_to hiện đang trống hoặc đĩa trên cùng của cọc c_to có kích thước lớn hơn d.

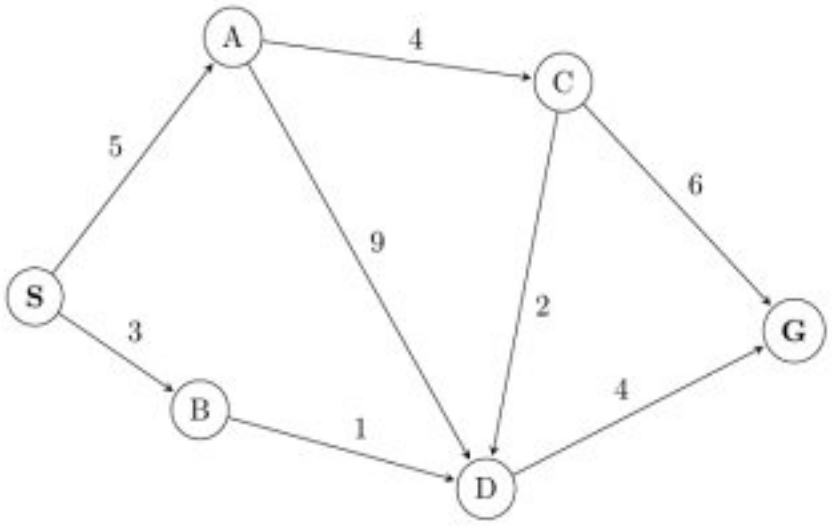
2.5. Goal Test

Hãy mô tả hàm goalTest(state): khi nào hàm trả về True?

Hàm goalTest(state) trả về True khi state = (3, 3, ..., 3), tức là toàn bộ n đĩa đã được chuyển thành công sang cọc thứ 3.

Bài 3. Thiết kế và phân tích Heuristic

Xét đồ thị dưới đây. Hãy thiết kế heuristic và phân tích ảnh hưởng của heuristic đến thuật toán A*.



Hình 3. Đồ thị dùng để thiết kế heuristic

Công thức A*: f(n) = g(n) + h(n), trong đó g(n) là chi phí thực tế từ S đến n và h(n) là chi phí ước lượng từ n đến đích.

3.1. Thiết kế heuristic admissible và consistent

Điền giá trị h(n) cho từng nút sao cho heuristic phù hợp với yêu cầu của nhóm.

Node	h(n)
S	8
A	10
B	5
C	6
D	4
G	0

3.2. Chạy A* với heuristic đã thiết kế

Explored Set:

S, B, D, G

Path: S -> B -> D -> G

Total Cost: 8

3.3. Heuristic admissible nhưng không consistent

Đề xuất một bộ giá trị h(n) admissible nhưng không consistent. Chỉ ra ít nhất một cạnh vi phạm điều kiện h(n) <= c(n,n') + h(n').

Ví dụ h(A) = 10, h(C) = 2. Cạnh A -> C có cost = 4. Vi phạm: h(A) = 10 > c(A,C) + h(C) = 4 + 2 = 6.

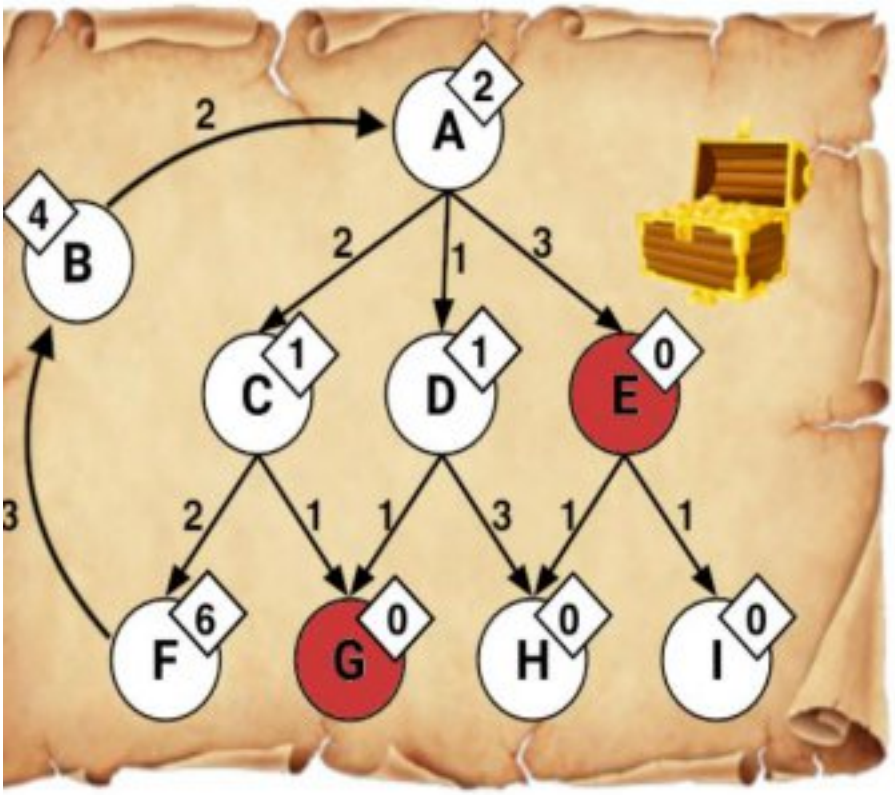
3.4. Câu hỏi mở rộng

Giải thích vì sao một heuristic consistent thì cũng admissible, giả sử h(G) = 0.

Vì consistent nghĩa là h(n) <= c(n, n') + h(n'). Áp dụng liên tiếp đến đích G với h(G) = 0, tổng chi phí các cạnh chính là đường đi thực tế g*(n), suy ra h(n) <= g*(n), thỏa mãn admissible.

Bài 4. Treasure Hunting (Tìm kho báu)

A là đảo xuất phát. Các nút màu đỏ là các đảo có kho báu; chỉ cần tới một trong các đảo đó. Mũi tên biểu diễn hành động có hướng; số trên cạnh là action cost. Giá trị trong hình thoi là heuristic ước lượng chi phí tối thiểu từ nút hiện tại tới một kho báu.



Hình 4. Bản đồ tìm kho báu (giữ nguyên hình gốc)

Quy tắc phá hòa: khi lấy phần tử khỏi frontier, nếu hòa thì chọn theo thứ

tụralphabet.

4.1. Depth-First Search

Explored Set:

A, C, G

Path returned:

A -> C -> G

4.2. Breadth-First Search

Explored Set:

A, C, D, E

Path returned:

A -> E

4.3. Uniform-Cost Search

Explored Set:

A, D, G

Path returned:

A -> D -> G

Cost:

2

4.4. Greedy Search

Explored Set:

A, D, E

Path returned:

A -> E

Cost: 3

4.5. A* Search

Explored Set:

A, D, G

Path returned:

A -> D -> G

Cost: 2

Bảng tổng hợp so sánh

Thuật toán	Tiêu chí chọn node	Path	Cost
DFS	Độ sâu	A -> C -> G	3
BFS	Độ sâu nhỏ nhất	A -> E	3
UCS	g(n)	A -> D -> G	2
Greedy	h(n)	A -> E	3
A*	g(n)+h(n)	A -> D -> G	2

Bài 5. Câu hỏi Đúng/Sai

Với mỗi phát biểu, hãy ghi Đúng hoặc Sai và giải thích ngắn gọn. Nếu sai, nên đưa ra phản ví dụ. **Câu 1.** Depth-first search luôn mở rộng ít nhất nhiều node bằng A* sử dụng một admissible heuristic. Đúng / Sai: **Sai**

Giải thích: DFS theo đuổi từng nhánh sâu, nếu may mắn chọn đúng nhánh có đích sớm nó có thể tìm thấy đích và dừng lại sau khi mở rộng rất ít node, ít hơn nhiều so với A*.

Câu 2. Giả sử một quân xe trong cờ vua có thể đi bất kỳ số ô theo hàng hoặc cột trong một nước đi và không thể nhảy qua quân khác. Manhattan distance là một admissible heuristic cho số nước đi nhỏ nhất từ ô A đến ô B. Đúng / Sai: **Sai**

Giải thích: Quân xe có thể đi nhiều ô trong 1 nước đi đi dọc theo hàng hoặc cột. Ví dụ di chuyển từ (0,0) đến (0,7) thực tế chỉ mất 1 nước đi, nhưng khoảng cách Manhattan là |0-0| + |0-7| = 7. Vì h(n) = 7 > g*(n) = 1 nên heuristic không admissible.

Câu 3. Euclidean distance là một admissible heuristic cho bài toán tìm đường của Pacman. Đúng / Sai: **Đúng**

Giải thích: Khoảng cách đường chim bay luôn nhỏ hơn hoặc bằng quãng đường di chuyển thực tế trên lưới.

Câu 4. Tổng của nhiều admissible heuristics vẫn luôn là một admissible heuristic.

Đúng / Sai: **Sai**

Giải thích: Tổng của hai heuristic admissible có thể vượt quá chi phí thực tế (ví dụ h1 + h2 > g*).

Câu 5. Admissibility của một heuristic cho A* kéo theo consistency.

Đúng / Sai: **Sai**

Giải thích: Admissibility không đảm bảo thỏa mãn bất đẳng thức tam giác của consistency.

Câu 6. A* với graph search luôn tối ưu nếu heuristic admissible.

Đúng / Sai: **Sai**

Giải thích: A* Graph Search chỉ đảm bảo tối ưu khi heuristic consistent (hoặc khi thuật toán cho phép mở lại/reopen các nút trong explored set). Nếu heuristic chỉ admissible mà không consistent và graph search không cho phép reopen nút, A* có thể trả về đường đi không tối ưu.

Tổng kết nhanh

Nhóm	Thuật toán	Hàm/tiêu chí chính
Uninformed Search	BFS / DFS / IDS	Không sử dụng heuristic
Cost-based Search	UCS	g(n)
Informed Search	Greedy	h(n)
Informed Search	A*	g(n)+h(n)

Ghi nhớ: A* dùng f(n) = g(n) +h(n).

---o0o---

(Hết)